

DAY 1 · MODULE 7 · 25 MIN

Lab Kickoff

Two agents, same job

What you'll build

- The same small docs assistant, three ways
- Part A — Prompt agent (created in the portal, no code to maintain)
- Part B — Hosted agent (deploy your own with azd; explore what Foundry manages)
- Part C — Your own code calling the Responses API (MAF in Python or C#)
- Ask each the same questions and compare behavior, latency, and where session state lives

Environment check — before you start

```
az login
az account show --query name -o tsv
azd version           # Azure Developer CLI (used from Day 3 onward)
python --version      # 3.11+
uv --version          # required for Python Labs
```

- FOUNDRY_PROJECT_ENDPOINT (from the portal tour)
- A model deployment name in that project (recommended: gpt-5.6-luna)

If anything fails, flag it now — get unblocked before the lab.

Repo and starter templates

```
git clone https://github.com/JimPiquant/Building-AI-Apps-and-Agents.git
cd Building-AI-Apps-and-Agents/labs/day1
```

```
# copy env template and fill in your values
cp .env.example .env
```

Day 1 lab lives under labs/day1/.

```
labs/day1/
├── README.md           ← the lab instructions
├── python/             ← Python starter templates (uv)
│   ├── pyproject.toml
│   ├── part_a_prompt_agent.py
│   ├── part_b_hosted_agent.py
│   └── part_c_responses_api.py
```

Success criteria — what 'done' looks like

- Part A runs — you created a Prompt agent in the Foundry portal (version 1.0) and your MAF app connects to it
- Part B runs — you deploy your own Hosted agent with azd and walk the portal to see what Foundry manages
- Part C runs — your MAF app calls the Responses API and handles multi-turn, streaming, and non-streaming
- A short reflection.md committed to your fork:
 - What does Foundry manage for you in Parts A and B that you'd handle yourself in Part C?
 - Which felt faster to iterate on?
 - Which would you pick for a shared cross-team production agent? Why?

How to get help

- Blocking issue? Post in the workshop channel
- Environment / RBAC / quota? Usually a 5-minute fix — flag in chat
- Code stuck? Pair up — lab is designed to be doable, not solo-only
- Instructor is on for questions during the lab

Common early gotchas (skim before you start)

- az login succeeded but MAF still 401s — need the Foundry User role on the Foundry RESOURCE (not the project). Fix in Access Control (IAM) on the resource.
- FOUNDRY_PROJECT_ENDPOINT not set — copy from .env.example, paste from portal
- Model deployment name mismatch — deployment name is not the same as model name
- Python — package missing — run `uv sync` from `labs/day1/python/` (do not use `pip install`)

Time expectations

- Part A → about 30 min (portal setup + connection)
- Part B → about 45 min (deploy your own Hosted agent via azd + connect + portal exploration)
- Part C → about 45 min (most of the code writing lives here; stretch deploy adds ~30 min)
- Reflection commit → ~10 min
- Total: ~2 hours of lab work

Going long? Ping an instructor — we'll help you scope down.

Preview: your capstone project

The workshop ends with a team capstone. Start thinking about your scenario — every day gives you a piece.

The format

- Teams of 2–3
- Starts at Day 5 close · demo day 2–3 weeks later
- Shared demo day — teams present to the workshop owners and participants
- ~15 min per team (10 demo + 5 Q&A)
- Coaching and feedback — no scoring
- Your team's choice of scenario

Required elements

- An MAF agent — Foundry-hosted or self-hosted
- Grounded in a Foundry-deployed model
- ≥1 Toolbox tool, MCP server, or function tool
- ≥1 Foundry IQ source or custom RAG
- Golden set of ≥10 items + captured eval score
- OTel traces in the portal or App Insights
- Architecture diagram + README with 30-day next steps

Start today: reflection question #4 asks what scenario your team would build — that's your capstone starter.

Takeaways

- Same underlying docs-assistant behavior — three hosting options with Foundry.
- Focus on feeling the difference — not just making the code run.
- The reflection is the deliverable, not the code.
- Start noodling on your capstone scenario this week.

Next: End of Day 1 live content. Have fun with the lab.